

Compte Rendu

Nom : OUMAST

Prenom: Zakaria

Groupe : Devoam202

Année Scolaire 2026/2027

Introduction

Dans le cadre du module **Kotlin (M202)** en Développement Mobile (filière DEVOIR), ce devoir explore la programmation asynchrone à travers les coroutines. La première partie modélise la gestion d'un restaurant (commandes, préparation parallèle en cuisine et paiements). La seconde partie traite de la gestion du stock d'un entrepôt et son suivi en temps réel à l'aide des Flow. Ce compte rendu présente l'architecture orientée objet mise en place et l'exécution non bloquante des différentes opérations.

EXERCICE 1:

Partie 1 : Prise de Commandes

```

class Commande( 8 Usages
    val id: Int,
    val plats: List<String>,
    val total: Double
) {
    fun afficherDetails() { 2 Usages
        println("Commande n°$id | Plats: $plats | Total: $total DH")
    }
}

class Serveur(val nom: String) { 3 Usages
    fun prendreCommande(plats: List<String>, prix: List<Double>): Commande { 1 Usage
        var totalDouble = 0.0
        for (p in prix) {
            totalDouble += p
        }
        val idUnique = Random.nextInt( from = 1, until = 1000)
        val commande = Commande(idUnique, plats, totalDouble)
        println("Le serveur $nom a pris la commande n°$idUnique.")
        return commande
    }

    fun afficherCommande(commandes: List<Commande>) {
        println("\n--- Commandes prises par $nom ---")
        for (c in commandes) {
            c.afficherDetails()
        }
    }
}
  
```

Figure 1: Implémentation de la classe Commande

La classe **Serveur** chargée de la prise et de l'affichage des commandes. La méthode **prendreCommande** calcule le total des prix, génère un ID unique et crée une Commande. La méthode **afficherCommande** parcourt une liste de commandes pour en afficher les détails. Cette étape constitue le point d'entrée fonctionnel de la gestion du restaurant.

Partie 2 : Préparation des Plats

```

class Cuisinier(val nom: String) { 3 Usages

    suspend fun preparerPlat(plat: String): String { 2 Usages
        println("$nom commence la préparation du plat : $plat")
        delay(timeMillis = 1000)
        println("$nom a terminé le plat : $plat")
        return plat
    }

    suspend fun preparerCommande(commande: Commande): List<String> = coroutineScope {
        val tasks = mutableListOf<Deferred<String>>()
        for (plat in commande.plats) {
            val task = async { preparerPlat(plat) }
            tasks.add(task)
        }
        tasks.awaitAll()
    }
}

class Cuisine(val cuisiniers: List<Cuisinier>) { 2 Usages

    suspend fun gererPreparationCommande(commande: Commande) = coroutineScope { 1 Usage
        println("\n[Cuisine] Début de la préparation de la commande n°${commande.id}...")
        val tasks = mutableListOf<Deferred<String>>()

        for (i in commande.plats.indices) {
            val plat = commande.plats[i]
            val cuisinier = cuisiniers[i % cuisiniers.size]
            val task = async { cuisinier.preparerPlat(plat) }
            tasks.add(task)
        }

        tasks.awaitAll()
        println("[Cuisine] Tous les plats de la commande n°${commande.id} sont prêts !")
    }
}
  
```

Figure 2 : Préparation Asynchrone en Cuisine

Les classes **Cuisinier** et **Cuisine** gèrent la préparation asynchrone des plats. La méthode **preparerPlat** simule un délai de cuisson non bloquant grâce à la fonction suspendue **delay**. Les méthodes **preparerCommande** et **gererPreparationCommande** lancent la préparation simultanée des plats. L'utilisation de **coroutineScope**, **async** et **awaitAll** garantit un traitement parallèle et optimisé en cuisine.

Partie 3 : Paiements

```

class Caisse { 2 Usages

    suspend fun traiterPaiement(commande: Commande): Boolean { 1 Usage
        println("\n[Caisse] Traitement du paiement pour la commande n°${commande.id}...")
        delay( timeMillis = 1000)

        val reussi = Random.nextInt( until = 100) < 80

        if (reussi) {
            println("[Caisse] Paiement de ${commande.total} DH validé avec succès !")
            return true
        } else {
            throw Exception("[Caisse] Échec du paiement pour la commande n°${commande.id} !")
        }
    }

    fun annulerPaiement(commande: Commande, job: Job) {
        job.cancel()
        println("[Caisse] Le paiement de la commande n°${commande.id} a été annulé.")
    }
}
  
```

Figure 3 : Gestion du Paiement Asynchrone

La classe **Caisse** responsable du traitement et de l'annulation des paiements. La méthode **traiterPaiement** simule une attente de 1s avec delay et une probabilité de réussite de 80%. En cas d'échec de la transaction, une exception est déclenchée pour être gérée par le système. La méthode **annulerPaiement** permet d'interrompre l'opération en annulant directement le Job de la coroutine associée.

Partie 4 : Gestion des Commandes avec Coroutines

```

class Restaurant( 1 Usage
    val serveurs: List<Serveur>,
    val cuisine: Cuisine,
    val caisse: Caisse
) {
    val commandes = mutableListOf<Commande>() 2 Usages

    suspend fun prendreCommandeEtTraiter(serveur: Serveur, plats: List<String>, prix: List<Double>) { 1 Usage
        val commande = serveur.prendreCommande(plats, prix)
        commandes.add(commande)

        cuisine.gererPreparationCommande(commande)

        try {
            caisse.traiterPaieement(commande)
        } catch (e: Exception) {
            println("Erreur : ${e.message}")
        }
    }

    fun afficherCommandesEnCours() { 1 Usage
        println("\n=== COMMANDES EN COURS ===")
        for (c in commandes) {
            c.afficherDetails()
        }
    }
}
  
```

Figure 5 : Coordination globale du Restaurant

Cette classe **Restaurant** centralise les serveurs, la cuisine et la caisse pour orchestrer le flux complet d'une commande. La fonction suspendue **prendreCommandeEtTraiter** enchaîne la prise de commande, sa préparation et son paiement de façon asynchrone. Un bloc try-catch intercepte et gère les éventuelles exceptions survenues lors du paiement en caisse. La méthode **afficherCommandesEnCours** permet de lister l'ensemble des commandes actuellement enregistrées.

Function Main() :

```
fun main() = runBlocking {  
    val serveurKarim = Serveur( nom = "Karim")  
    val chefAli = Cuisinier( nom = "Chef Ali")  
    val chefSami = Cuisinier( nom = "Chef Sami")  
  
    val cuisine = Cuisine( cuisiniers = listOf(chefAli, chefSami))  
    val caisse = Caisse()  
    val restaurant = Restaurant( serveurs = listOf(serveurKarim), cuisine, caisse)  
  
    val listePlats = listOf("Tajine de Poulet", "Salade Marocaine", "Jus d'Orange")  
    val listePrix = listOf(60.0, 20.0, 15.0)  
  
    restaurant.prendreCommandeEtTraiter(serveurKarim, listePlats, listePrix)  
    restaurant.afficherCommandesEnCours()  
}
```

Figure 6: Exécution Principale du Restaurant

La fonction main utilisant le constructeur **runBlocking** pour faire la jonction avec le monde asynchrone des coroutines. Elle instancie l'ensemble des acteurs du système : le serveur Karim, les cuisiniers Chef Ali et Chef Sami, la cuisine et la caisse. Une commande concrète est préparée en définissant une liste de plats ("**Tajine de Poulet**", "**Salade Marocaine**", "**Jus d'Orange**") et leurs prix respectifs. Le programme déclenche ensuite l'enchaînement complet du traitement (prise, préparation et paiement) puis affiche le récapitulatif des commandes.

EXERCICE 2 :

Partie 1 : Produits et Stock :

```

class Stock { 3 Usages
    val produits = mutableMapOf<Int, Produit>() 4 Usages

    fun ajouterProduit(produit: Produit) { 1 Usage
        produits[produit.id] = produit
        println("Produit '${produit.nom}' ajouté au stock.")
    }

    suspend fun ajouterQuantite(idProduit: Int, quantite: Int) {
        delay( timeMillis = 500) // Simulation d'une opération asynchrone
        val produit = produits[idProduit]
        if (produit != null) {
            produit.quantite += quantite
            println("+ $quantite ajoutée(s) au produit '${produit.nom}'. Nouvelle quantité: ${produit.quantite}")
        } else {
            println("Produit ID $idProduit introuvable.")
        }
    }

    suspend fun retirerQuantite(idProduit: Int, quantite: Int) { 2 Usages
        delay( timeMillis = 500) // Simulation d'une opération asynchrone
        val produit = produits[idProduit]
        if (produit == null) {
            throw Exception("Produit ID $idProduit introuvable !")
        }
        if (produit.quantite < quantite) {
            throw Exception("Stock insuffisant pour '${produit.nom}' ! (Demandé: $quantite, Disponible: ${produit.quantite})")
        }
        produit.quantite -= quantite
        println("- $quantite retirée(s) du produit '${produit.nom}'. Quantité restante: ${produit.quantite}")
    }

    fun afficherStock() { 1 Usage
        println("\n=== ÉTAT ACTUEL DU STOCK ===")
        for (produit in produits.values) {
            produit.afficherDetails()
        }
    }
}
  
```

Figure 1 : Gestion du Stock Asynchrone

la classe **Stock** qui gère l'inventaire des produits via une **MutableMap**. Les fonctions suspendues **ajouterQuantite** et **retirerQuantite** simulent des opérations asynchrones avec **delay**. La méthode de retrait lève des exceptions si le produit est introuvable ou si la quantité en stock est insuffisante. La fonction **afficherStock** parcourt la collection pour afficher l'état actuel de tous les produits enregistrés.

Partie 2 : Commandes Clients

```

class GestionnaireCommandes(val stock: Stock) { 2 Usages
    val commandes = mutableListOf<CommandeClient>()

    suspend fun traiterCommande(commande: CommandeClient) { 1 Usage
        println("\n[Traitement] Début de la commande n°${commande.idCommande}...")
        for (item in commande.produits) {
            val idProduit = item.first
            val quantiteDemandee = item.second
            try {
                stock.retirerQuantite(idProduit, quantiteDemandee)
            } catch (e: Exception) {
                println("Erreur commande n°${commande.idCommande} : ${e.message}")
            }
        }
        println("[Traitement] Fin de la commande n°${commande.idCommande}")
    }

    suspend fun gererCommandes(listeCommandes: List<CommandeClient>) = coroutineScope {
        for (cmd in listeCommandes) {
            launch {
                traiterCommande(cmd)
            }
        }
    }
}
  
```

Figure 2 : Gestionnaire des Commandes Clients

Cette classe **GestionnaireCommandes** permet de traiter les commandes clients de manière asynchrone et parallèle. La fonction **traiterCommande** parcourt les articles demandés et retire les quantités du stock tout en gérant les exceptions. La méthode **gererCommandes** utilise `coroutineScope` et `launch` pour exécuter plusieurs commandes simultanément sans bloquer l'application. Elle garantit ainsi un traitement concurrent et sécurisé du stock pour l'ensemble des requêtes clients.

Partie 3 : Opérations Asynchrones sur le Stock

```

class Entrepot( 1 Usage
    val stock: Stock,
    val gestionnaireCommandes: GestionnaireCommandes
) {
    suspend fun ajouterProduitAuStock(produit: Produit) = coroutineScope {
        launch {
            stock.ajouterProduit(produit)
        }
    }

    suspend fun retirerProduitDuStock(idProduit: Int, quantite: Int) = coroutineScope {
        launch {
            try {
                stock.retirerQuantite(idProduit, quantite)
            } catch (e: Exception) {
                println("Erreur Entrepôt : ${e.message}")
            }
        }
    }

    suspend fun gererInventaire() = coroutineScope {
        launch {
            println("\n--- INVENTAIRE EN COURS ---")
            delay( timeMillis = 300)
            stock.afficherStock()
            println("--- FIN DE L'INVENTAIRE ---\n")
        }
    }
}
  
```

Figure 3: Opérations Asynchrones de l'Entrepôt

Cette classe **Entrepot** regroupe les opérations globales sur le stock et la gestion des commandes. Les fonctions **ajouterProduitAuStock** et **retirerProduitDuStock** utilisent **launch** pour s'exécuter en arrière-plan sans bloquer. Un bloc try-catch dans le retrait permet d'intercepter les erreurs de stock insuffisant ou de produit introuvable. La méthode **gererInventaire** simule la vérification du stock en parallèle des autres tâches grâce à un délai non bloquant

Partie 4 : Flux d'Événements

```
data class EvenementStock(val message: String) 3 Usages

class GestionnaireEvenements { 1 Usage
    private val _fluxEvenements = MutableSharedFlow<EvenementStock>() 2 Usages
    val fluxEvenements: SharedFlow<EvenementStock> = _fluxEvenements

    suspend fun emettreEvenement(message: String) {
        _fluxEvenements.emit(value = EvenementStock(message = message))
    }
}
```

Figure 4: Gestion des Événements Flow

la classe **GestionnaireEvenements** et le modèle **EvenementStock** pour la réactivité du stock. Elle utilise un **MutableSharedFlow** privé pour émettre les événements de mise à jour en temps réel. Le flux public en lecture seule **fluxEvenements** sépare la gestion de l'émission et de l'observation. La fonction suspendue **emettreEvenement** transmet les messages aux observateurs de manière bloquant.

Conclusion

Ce devoir a permis d'approfondir la gestion de la concurrence en Kotlin grâce aux coroutines et aux Flows. Les cas pratiques développés démontrent une maîtrise de l'exécution parallèle, de la résilience aux erreurs et de la mise à jour réactive des données. Ces acquis sont primordiaux pour garantir une expérience utilisateur fluide et sans ralentissement dans le développement mobile.

[MyGithubLink](#)